

PRODUCT MODULE PDF EDITION

Module Catalog

Stable owner cards for every product module and root-level product surface.

Core rule: 00-05 form the document ingestion and materialization mainline; 06-08 control it; frontend, SampleDB, tools and installer are product or release surfaces.

Contents

A maintainer-oriented map of module ownership, dependencies, outputs, boundaries, and debugging start points.

- First read: product surface map 3
- Catalog Summary 4
- 00 - Orchestrator 5
- 01 - Optimizer 7
- 02 - Interpreter 9
- 03 - Validator 11
- 04 - Normalizer 13
- 05 - Corpus Builder 15
- 06 - Edit Suite 18
- 07 - MCP Server 20
- 08 - Semantic Control Kernel 22
- Client Frontend 25
- SampleDB 29
- Root Tooling 30
- Installer And Packaging 32
- Cross-References, Not Module Cards 34
- Fast Module Selection Guide 35
- Related Chapters 36

Recommended first pass: read the product surface map, scan the catalog summary, then jump to the module card whose ownership boundary matches the problem.

First read: product surface map

This catalog gives every product module and root-level product surface a stable handover card. It is meant for maintainers who need to know where to start, what a module owns, what it consumes, what it produces, and what it must not be made responsible for.

The numbered modules are not equal kinds of things. 00 through 05 form the document ingestion and materialization mainline. 06, 07 and 08 are control surfaces around that mainline. Client Frontend is the local browser product surface. SampleDB, tools and installer are root-level product or release surfaces, not pipeline stages.

One sentence to remember: the numbered modules are the operational spine; the frontend, Kernel, MCP Server, Edit Suite, SampleDB, tools, and installer are surfaces around that spine with deliberately narrower responsibilities.

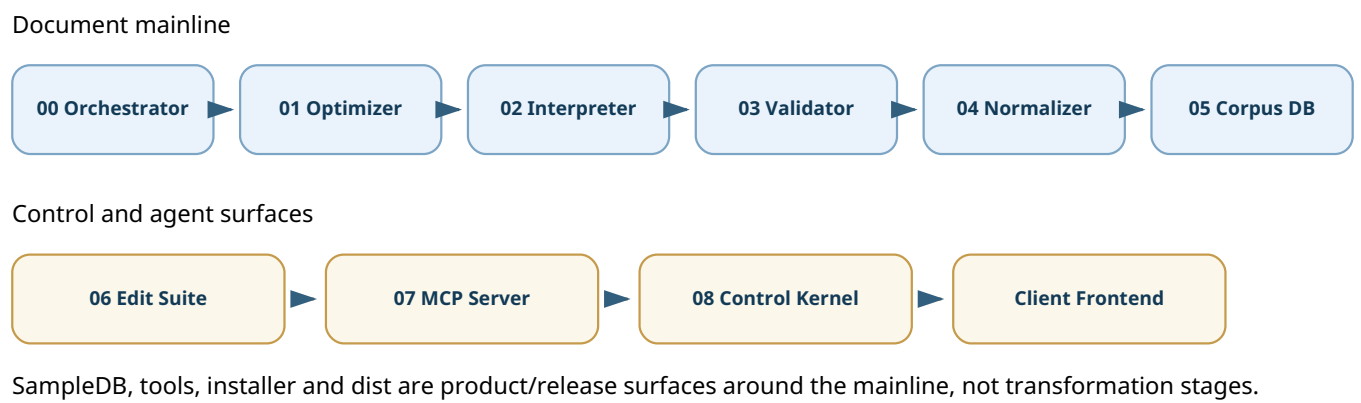


Figure 1. Minimum orientation: pipeline modules transform documents; control and frontend surfaces coordinate, expose, or package the work.

The shortest useful reading path

1. Use the catalog summary to identify the likely owner.
2. Open that module card and check **Role**, **Owns**, and **Must Not Own** before changing code.
3. Use **Consumes** and **Produces** to trace adjacent module contracts.
4. Use **Test Orientation** and the fast selection guide when debugging failures.

Catalog Summary

Surface	Type	Primary Role
00 - Orchestrator	Python module, runtime host	Runs the document pipeline and owns runtime coordination.
01 - Optimizer	Python module, pipeline stage	Turns source files into page assets and raw extraction payloads.
02 - Interpreter	Python module, pipeline stage	Turns request-enriched page/document input into structured LLM interpretation.
03 - Validator	Python module, pipeline stage	Validates structured output and marks hard/soft failure evidence.
04 - Normalizer	Python module, pipeline stage and release owner	Maps structured output into the active Semantic Release and owns release authoring.
05 - Corpus Builder	Python module, DB/materialization owner	Creates and maintains the Corpus DB, search, embeddings, merge, rebuild, Base Graph and ontology schema.
06 - Edit Suite	Python desktop app	Advanced owner-edit shell for visible config and edit surfaces.
07 - MCP Server	Python local stdio server	Local MCP tool bridge and owner-contract delegation layer.
08 - Semantic Control Kernel	Python control module	Workflow state machine for Taxonomy Agent operations.
Client Frontend	Node/Vite local browser app	Chat/config UI, Query Agent, Ontology Agent and Taxonomy Agent surface.
SampleDB	Product payload	Bundled demo Artifact Tree and Corpus DB.
tools	Root tooling	Runtime build, installer staging and test dispatch helpers.
installer / dist	Packaging surface	Windows all-in-one installer source and generated release output.

00 - Orchestrator

MODULE KEY orchestrator	DISPLAY NAME Orchestrator
RUNTIME bundled Python runtime under runtime/python	LAUNCHER MODULE orchestrator
PRIMARY CONTRACT orchestrator.orchestrator_contract	ADMIN CONTRACT orchestrator.admin_contract

Role

The Orchestrator is the document-mainline runtime host. It owns pipeline startup, module discovery, stage scheduling, artifact tree selection, runtime settings, provider/model injection for ingestion, batch manifests, debug runs and final pipeline disposition.

It is the correct owner for "the ingestion run itself", not for every product capability. Taxonomy creation, ontology mining and conversational corpus work live around it.

Main Actions

run

reset

reset_pipeline_logs

embeddings

activate_corpus_context

inspect_source_document_sample

kernel_llm_runtime_profile

kernel_llm_generate

healthcheck

create_artifact_tree

validate_artifact_tree

create_pipeline_batch_manifest

finalize_pipeline_batch_manifest

Owns

- module registry and runtime resolution
- pipeline stage scheduling
- intake/runtime semantics/request enrichment orchestration
- runtime credentials and model settings for pipeline calls
- pipeline run state and final disposition
- Artifact Tree selection and creation path
- pipeline batch manifests
- debug-host behavior for module runs
- Kernel LLM bridge into the configured runtime profile

Consumes

- registered sibling modules 01 through 05

- runtime settings and credentials
- source files in the selected Artifact Tree
- active Semantic Release and Corpus Builder context when ingestion runs

Produces

- run-scoped pipeline state
- `Documents/*` artifacts for successful pages/documents
- `Error Cases/*` artifacts for failed or weak cases
- pipeline batch manifests
- Orchestrator logs and debug outputs

Must Not Own

- Corpus DB schema
- Semantic Release authoring truth
- module-local implementation internals
- Client Frontend session state
- Kernel workflow state
- MCP transport

Implementation Anchors

- `00 - Orchestrator/module-manifest.json`
- `00 - Orchestrator/module-registry.json`
- `00 - Orchestrator/config/execution_policy.json`
- `00 - Orchestrator/orchestrator/pipeline/`
- `00 - Orchestrator/orchestrator/orchestrator_contract/`
- `00 - Orchestrator/orchestrator/admin_contract/`

Test Orientation

Use the root test runner for full suite checks:

```
run-dev-tests.bat --module "00 - Orchestrator"
```

Start here when pipeline stage counts, final notices, startup checks, health checks, runtime credentials or artifact publication are wrong.

01 - Optimizer

MODULE KEY optimizer	DISPLAY NAME Optimizer
RUNTIME bundled Python runtime under runtime/python	LAUNCHER MODULE ingestion_layer_vision
PRIMARY CONTRACT ingestion_layer_vision.orchestrator_contract	

Role

The Optimizer is the first transformation stage. It classifies and extracts supported source files into page assets and raw extraction payloads. It has a vision profile and a file profile, but the product-facing manifest exposes the Optimizer as one module.

The Optimizer should keep its output as non-semantic as possible. It prepares evidence for downstream interpretation; it should not decide what a document means.

Main Actions

classify_document extract_document healthcheck scan_debug_input debug_run

Owns

- document/file profile preparation
- page image generation or page asset references
- raw extraction payloads
- OCR/extraction route selection
- Optimizer debug runs
- local extraction plugins and optional Outlook fallback behavior

Consumes

- source files from Orchestrator intake
- runtime semantic assets when needed
- Orchestrator-injected OCR/model configuration for image-like extraction
- bundled file conversion/runtime dependencies

Produces

- raw extraction JSON
- page image paths or page asset paths
- document/page classification output
- Optimizer debug artifacts

Must Not Own

- final document semantics
- projection routing
- validation policy
- normalized payloads
- Corpus DB materialization
- provider secret persistence
- product GUI behavior

Implementation Anchors

- 01 - Optimizer/module-manifest.json
- 01 - Optimizer/ingestion_layer_vision/
- 01 - Optimizer/ingestion_layer_file/
- 01 - Optimizer/optimizer_ocr/
- 01 - Optimizer/plugins/
- 01 - Optimizer/config/rules/

Test Orientation

```
run-dev-tests.bat --module "01 - Optimizer"
```

Start here when a source file fails to open, render, split, classify or produce usable raw/page artifacts.

02 - Interpreter

MODULE KEY interpreter	DISPLAY NAME Interpreter
RUNTIME bundled Python runtime under runtime/python	LAUNCHER MODULE llm_interpreter
PRIMARY CONTRACT llm_interpreter.orchestrator_contract	

Role

The Interpreter turns a request-enriched page or document unit into structured LLM interpretation. It answers the pipeline question "what do you see?" while preserving enough structure for the Validator and Normalizer to work with.

It also provides the generic generate_llm bridge used by the Semantic Control Kernel through the Orchestrator-hosted runtime path.

Main Actions

- interpret_document
- healthcheck
- debug_run
- generate_llm

Owns

- Interpreter prompt bundles
- provider abstraction for Interpreter calls
- structured output generation
- debug runs for interpretation
- model response validation local to Interpreter output shape

Consumes

- request-enriched artifacts from Orchestrator
- raw/page evidence produced by Optimizer
- Orchestrator-injected provider credentials and model settings
- runtime profile for Kernel LLM calls when called through Orchestrator

Produces

- structured interpretation payloads
- LLM request/response artifacts
- Interpreter debug outputs
- generic LLM responses for Kernel-owned isolated calls

Must Not Own

- provider credential storage
- Optimizer raw extraction
- Validator pass/fail policy
- Normalizer taxonomy mapping
- Corpus DB writes
- UI/session state

Implementation Anchors

- 02 - Interpreter/module-manifest.json
- 02 - Interpreter/llm_interpreter/orchestrator_contract/
- 02 - Interpreter/llm_interpreter/interpreter/
- 02 - Interpreter/llm_interpreter/providers/
- 02 - Interpreter/llm_interpreter/prompts/
- 02 - Interpreter/llm_interpreter/models/

Test Orientation

```
run-dev-tests.bat --module "02 - Interpreter"
```

Start here when the structured interpretation is semantically wrong before validation or normalization touches it.

03 - Validator

MODULE KEY validator	DISPLAY NAME Validator Vision
RUNTIME bundled Python runtime under runtime/python	LAUNCHER MODULE validator_vision
PRIMARY CONTRACT validator_vision.orchestrator_contract	EDIT CONTRACT validator_vision.edit_contract

Role

The Validator checks Interpreter output against validation policy and available evidence. It creates validation reports, hard failure signals and review flags. It is not meant to make the Machine perfect; it makes uncertainty visible and keeps silent drift from moving downstream unnoticed.

Main Actions

- validate_document
- healthcheck
- debug_run

Owns

- validation reports
- profile-specific validation logic
- numeric/date/value survival checks
- soft review flags and review reasons
- Validator edit surfaces

Consumes

- structured output from Interpreter
- raw/page evidence from Optimizer
- validation settings from Validator config/edit surfaces

Produces

- validation reports
- hard fail evidence for Error Cases
- review flags and review reasons

- debug validation artifacts

Must Not Own

- extraction
- LLM interpretation
- taxonomy/projection authoring
- normalized payload materialization
- Corpus DB schema
- provider credentials

Implementation Anchors

- 03 - Validator/module-manifest.json
- 03 - Validator/validator_vision/orchestrator_contract/
- 03 - Validator/validator_vision/edit_contract/
- 03 - Validator/validator_vision/validator/
- 03 - Validator/config/

Test Orientation

```
run-dev-tests.bat --module "03 - Validator"
```

Start here when a bad value should have been rejected, a good value is rejected, or a review flag is too noisy or too weak.

04 - Normalizer

MODULE KEY normalizer	DISPLAY NAME Normalizer
RUNTIME bundled Python runtime under runtime/python	LAUNCHER MODULE normalizer_vision
PRIMARY CONTRACT normalizer_vision.orchestrator_contract	

Role

The Normalizer maps flexible structured interpretation into the active Semantic Release. It owns taxonomy and projection authoring, release compilation, release validation, runtime semantic assets and normalized canonical payloads.

This is the semantic contract owner. If the taxonomy or projection is wrong, start here before changing Corpus Builder or agents.

Main Actions

normalize_document

build_projection_catalog

build_runtime_semantic_assets

publish_semantic_release

list_default_blueprints

export_default_blueprint_release

create_zero_shot_working_release

healthcheck

debug_run

Owns

- taxonomy source authoring
- projection authoring
- Semantic Release package creation and validation
- runtime semantic assets
- Normalizer prompt bundle and normalization behavior
- normalized payload contract

Consumes

- structured output from Interpreter
- validation context from Validator path
- user/sample intent through Kernel-owned workflows
- Orchestrator-injected provider settings for LLM-assisted authoring

Produces

- normalized payloads
- projection catalogs
- runtime semantic assets
- Semantic Release packages
- default/custom taxonomy and projection artifacts

Must Not Own

- Corpus DB schema
- active DB attachment state
- page images
- Orchestrator runtime state
- provider secret storage
- Kernel workflow dialogs or resume state

Implementation Anchors

- 04 - Normalizer/module-manifest.json
- 04 - Normalizer/normalizer_vision/orchestrator_contract/
- 04 - Normalizer/normalizer_vision/normalizer/
- 04 - Normalizer/normalizer_vision/semantic_release/
- 04 - Normalizer/normalizer_vision/source_authoring/
- 04 - Normalizer/normalizer_vision/runtime_semantic_assets/
- 04 - Normalizer/config/prompt_bundle.json

Test Orientation

```
run-dev-tests.bat --module "04 - Normalizer"
```

Start here when a release, projection, normalized field, row code, promotion rule or release fingerprint is wrong.

05 - Corpus Builder

MODULE KEY corpus_builder	DISPLAY NAME Corpus Builder Vision
RUNTIME bundled Python runtime under runtime/python	LAUNCHER MODULE corpus_builder
PRIMARY CONTRACT corpus_builder.orchestrator_contract	

Role

The Corpus Builder owns the SQLite corpus. It loads normalized payloads and related artifacts into a self-contained DB, manages active release binding, creates schema, writes page images, evidence, fields, rows, promotions, entities, embeddings, source-document structure, Base Graph rows and ontology schema.

It also owns corpus merge, rebuild and search/export/stats surfaces.

Main Actions

load_document

activate_semantic_release

activate_corpus_context

create_empty_corpus_db

reset_active_corpus_db

create_and_activate_new_corpus_db

activation_preflight

generate_embeddings

healthcheck

scan_debug_input

debug_run

semantic_status

read_active_semantic_release

load_semantic_release

semantic_audit

backfill_stale

merge_preflight

merge_corpus_databases

validate_artifact_tree

read_database_analysis_evidence

inspect_latest_pipeline_batch

extract_sample_files_for_reingest

restore_pipeline_batch_originals

cleanup_pipeline_batch_materialization

reingest_pipeline_batch

multi_source_merge_preflight

multi_source_merge_databases

write_merge_reconciliation_manifest

backfill_sql_from_merge_artifacts

search

stats

export

preview_rebuild_from_artifacts

rebuild_from_artifacts

create_and_rebuild_new_corpus_db

basic_relation_mining

Owns

- Corpus DB schema
- schema migration and compatibility checks
- document/page materialization
- document_page_images
- evidence atoms and candidate evidence
- extracted fields and rows

- promotions and slot candidates
- entities and semantic evidence links
- embeddings and embedding chunks
- source-document structure
- deterministic Base Graph mining
- ontology schema
- additive merge and artifact-preserving filled merge
- rebuild from artifacts

Consumes

- normalized payloads from Normalizer
- structured output and validation reports for materialization context
- Optimizer page images and raw extracts
- active Semantic Release package
- optional embedding provider capability through Orchestrator

Produces

- SQLite Corpus DB
- DB-local page image evidence rows
- search/FTS/read views
- embedding rows/chunks
- Base Graph/source-document rows
- ontology schema and ontology validation surfaces
- merge/rebuild reports and manifests

Must Not Own

- taxonomy/projection authoring truth
- Orchestrator UI state
- provider secret persistence
- Kernel workflow state
- Client Frontend chat behavior

Implementation Anchors

- 05 - Corpus Builder/module-manifest.json
- 05 - Corpus Builder/corpus_builder/orchestrator_contract/
- 05 - Corpus Builder/corpus_builder/database/
- 05 - Corpus Builder/corpus_builder/loader/
- 05 - Corpus Builder/corpus_builder/semantic_release/

- 05 - Corpus Builder/corpus_builder/embeddings/
- 05 - Corpus Builder/corpus_builder/search/
- 05 - Corpus Builder/corpus_builder/ontology/basic_relation_mining.py
- 05 - Corpus Builder/README.operations.md

Test Orientation

```
run-dev-tests.bat --module "05 - Corpus Builder"
```

Start here when DB rows, views, page images, embeddings, source-document structure, Base Graph, ontology schema, merge or rebuild behavior is wrong.

06 - Edit Suite

MODULE KEY <code>edit_suite</code>	DISPLAY NAME Edit Suite
RUNTIME bundled Python runtime under <code>runtime/python</code>	LAUNCHER MODULE <code>edit_suite</code>
PRIMARY CONTRACT <code>edit_suite.orchestrator_contract</code>	

Role

The Edit Suite is an advanced Windows desktop shell for owner-provided edit surfaces. It discovers sibling modules, reads their edit contracts and renders editable or read-only surfaces for configuration and readiness work.

It is not a pipeline runner and not a debug host.

Main Actions

healthcheck

Owns

- Edit Suite UI state
- registry discovery cache
- owner surface bundle cache
- suite-local confirmation artifacts
- temporary edit-contract I/O directories
- background UI jobs for owner actions

Consumes

- sibling owner edit contracts
- owner `describe_surfaces`, `read_surface`, `validate_surface`, `write_surface`
- `read_bundle` where the owner supports it

Produces

- visible edit UI

- suite-local state/cache under `state/`
- validation/write calls through owner contracts

Must Not Own

- foreign module raw state
- pipeline execution
- module debug hosting
- Semantic Release truth outside owner contracts
- Corpus DB mutation

Implementation Anchors

- `06 - Edit Suite/module-manifest.json`
- `06 - Edit Suite/README.md`
- `06 - Edit Suite/edit_suite/registry/`
- `06 - Edit Suite/edit_suite/surfaces/`
- `06 - Edit Suite/edit_suite/ui/`
- `06 - Edit Suite/edit_suite/orchestrator_contract/`

Test Orientation

```
run-dev-tests.bat --module "06 - Edit Suite"
```

Start here when an edit surface does not render, cache/discovery is stale, or owner edit-contract calls are not shown correctly in the desktop UI.

07 - MCP Server

MODULE KEY mcp_server	DISPLAY NAME MCP Server
RUNTIME bundled Python runtime under runtime/python	LAUNCHER MODULE mcp_server
PRIMARY CONTRACT mcp_server.orchestrator_contract	EDIT CONTRACT mcp_server.edit_contract

Role

The MCP Server is the local stdio tool bridge. It exposes a tool catalog, validates tool arguments, applies permission policy and delegates real work to the module that owns the action. It also bridges the Semantic Control Kernel to the Taxonomy Agent.

It is transport and delegation, not a second business-logic host.

Main Actions

- serve
- healthcheck

Owns

- local stdio JSON-RPC transport
- MCP tool catalog
- tool schema validation
- permission level checks
- owner-contract delegation
- Kernel bridge subprocess configuration
- MCP-local support monitor state

Consumes

- owner contracts from Orchestrator, Optimizer, Validator, Normalizer and Corpus Builder
- Semantic Control Kernel contract subprocess
- MCP permission config

Produces

- MCP tool responses
- support monitor events/reports
- temporary owner-contract call state
- Kernel bridge calls for agent workflow tools

Must Not Own

- Corpus DB truth
- Semantic Release truth
- Kernel workflow state
- Client Frontend UI state
- direct cross-module writes

Implementation Anchors

- 07 - MCP Server/module-manifest.json
- 07 - MCP Server/README.md
- 07 - MCP Server/mcp_server/server.py
- 07 - MCP Server/mcp_server/tools.py
- 07 - MCP Server/mcp_server/semantic_control_kernel_client.py
- 07 - MCP Server/mcp_server/tool_handlers_semantic_control_kernel.py
- 07 - MCP Server/config/agent_permissions.json
- 07 - MCP Server/config/semantic_control_kernel_bridge.json

Test Orientation

```
run-dev-tests.bat --module "07 - MCP Server"
```

Start here when a tool is missing from the MCP catalog, the local MCP stdio server fails, permission policy blocks unexpectedly, or Kernel bridge calls do not reach 08 - Semantic Control Kernel .

08 - Semantic Control Kernel

MODULE KEY semantic_control_kernel	DISPLAY NAME Semantic Control Kernel
RUNTIME bundled Python runtime under runtime/python	LAUNCHER MODULE semantic_control_kernel
PRIMARY CONTRACT semantic_control_kernel.orchestrator_contract	

Role

The Semantic Control Kernel is the headless workflow brain behind the Taxonomy Agent. It owns workflow selection, dialogs, blockers, progress events, mirror events, recovery, resume state, receipts and adapter orchestration.

The Kernel exists because long creation and merge workflows need durable state. They cannot safely live only in an agent chat transcript.

Main Actions

empty_database_no_semantic_release

empty_database_default_taxonomy_no_projections

empty_database_default_taxonomy_default_projections

empty_database_default_taxonomy_custom_projections

empty_database_custom_taxonomy_no_projections

empty_database_custom_taxonomy_custom_projections

manual_pipeline_run

database_merge_additive_only

database_rebuild_from_artifacts

create_custom_taxonomy_path

create_custom_projection_path

reset_database

kernel_status

kernel_resume_state

kernel_continue_resumable_workflow

kernel_cancel_active_run

Owns

- workflow semantics
- user interaction contracts
- confirmations and pending interactions
- progress/mirror/recovery events
- resume options
- workflow receipts
- support bundles and debug traces
- adapter call state
- Kernel-owned state repository under state/

Consumes

- MCP bridge calls from the Client Frontend Taxonomy Agent
- Orchestrator runtime and Kernel LLM profile
- Corpus Builder DB primitives for create/reset/merge/rebuild
- Normalizer release authoring/materialization primitives
- Orchestrator pipeline run primitives

Produces

- workflow state
- progress events
- user interaction requests
- resume state
- receipts/final notices
- recovery/support information
- owner adapter calls

Must Not Own

- UI rendering
- MCP stdio transport
- Corpus DB schema
- Semantic Release primitive implementation
- provider credential storage
- document transformation stages

Implementation Anchors

- 08 - Semantic Control Kernel/module-manifest.json
- 08 - Semantic Control Kernel/README.md
- 08 - Semantic Control Kernel/semantic_control_kernel/orchestrator_contract.py
- 08 - Semantic Control Kernel/semantic_control_kernel/mcp_contract.py
- 08 - Semantic Control Kernel/semantic_control_kernel/surface/agent_tools.py
- 08 - Semantic Control Kernel/semantic_control_kernel/surface/mcp_tool_schemas.py
- 08 - Semantic Control Kernel/semantic_control_kernel/services/agent_workflow_dispatcher.py
- 08 - Semantic Control Kernel/semantic_control_kernel/repository/
- 08 - Semantic Control Kernel/semantic_control_kernel/workflows/

Test Orientation

```
run-dev-tests.bat --module "08 - Semantic Control Kernel"
```

Start here when a Taxonomy Agent workflow asks the wrong question, blocks incorrectly, resumes badly, loses progress events, misses final notices, or misclassifies recovery state.

Client Frontend

TYPE Node ESM + Vite local browser app	RUNTIME Node.js
NORMAL ENTRY POINT node server/index.js	CONFIG ENTRY POINT node server/index.js --config
NORMAL BIND 127.0.0.1, configured port or 3000	CONFIG BIND 127.0.0.1:3001

Role

The Client Frontend is the normal browser-facing product surface. It hosts the chat UI, config UI, credentials/OAuth flows, source list, page image viewer, Kernel progress/dialog rendering and the three agent surfaces.

It is not a document transformation stage.

Main Scripts

- npm run build
- npm run start
- npm run config
- npm run minimal
- npm run runtimes:check

Main HTTP Routes

Route	Surface
GET /	main browser app
GET /config	config page
GET /assets/*	frontend assets
GET /api/v2/health	health/config status
POST /api/v2/chat	Query Agent
POST /api/v2/pipeline-manager/chat	Taxonomy Agent code path
POST /api/v2/ontology-agent/chat	Ontology Agent
/api/chat/*	Query Agent history
/api/pipeline-manager/*	Taxonomy Agent history and Kernel bridge routes
/api/ontology-agent/*	Ontology Agent history
/config/api/*	config/model/prompt/credential APIs
/config/oauth/*	OAuth login/callback/logout

The route name `pipeline-manager` is retained in code and API paths for compatibility. The user-facing name is Taxonomy Agent.

Agent Surfaces

Query Agent

- read-only corpus analyst
- opens the active DB in read-only mode
- can use SQL, compact/full document view tools, provenance tools, semantic search, coverage snapshots, source-document readers, ontology lens readers and restricted read-only workbench
- must not write to the DB

Ontology Agent

- reads the same corpus as the Query Agent
- uses the same compact document view repository surface, but not the Query Agent workbench
- adds `basic_relation_mining`
- adds `sql_batch_execute`
- can write only allowlisted ontology/relation/source-structure tables through preflight, transaction, validation, edit logging and embedding refresh
- must not edit base documents, extracted fields/rows, evidence atoms, promotions or normal embedding tables directly

Taxonomy Agent

- user-facing agent for Kernel workflow selection and explanation
- discovers and launches the local MCP Server under the configured pipeline root
- validates the Kernel tool surface
- calls Kernel workflow tools through MCP
- must not become the Kernel or collect Kernel-owned workflow values directly

Owns

- browser UI
- config UI
- provider/model selection UI
- credential/OAuth server-side app-home state
- chat sessions and history
- frontend policy prompts
- Query/Ontology/Taxonomy Agent prompts and runtime policies
- source-link resolution UI
- Kernel progress/dialog rendering

Consumes

- active Corpus DB path
- configured Ontology Machine/pipeline root
- provider credentials and OAuth state
- MCP Server process
- Kernel event bridge
- Corpus DB read/write repositories depending on agent role

Produces

- chat responses and histories
- source list and page image viewer state
- frontend config files
- credential readiness/status reports
- Kernel interaction submissions
- ontology edit batches through controlled tools

Must Not Own

- pipeline transformation behavior
- Corpus DB schema
- Semantic Release authoring primitive logic
- Kernel workflow state
- MCP transport internals
- raw provider secrets in browser payloads

Implementation Anchors

- `Client Frontend/package.json`
- `Client Frontend/server/index.js`
- `Client Frontend/client_frontend/http/`
- `Client Frontend/client_frontend/min_agent/`
- `Client Frontend/client_frontend/min_agent/document_repository.js`
- `Client Frontend/client_frontend/min_agent/document_view_support.js`
- `Client Frontend/client_frontend/ontology_agent/`
- `Client Frontend/client_frontend/pipeline_agent/`
- `Client Frontend/client_frontend/config/`
- `Client Frontend/client_frontend/credentials/`
- `Client Frontend/client_frontend/frontend_policy/`
- `Client Frontend/client_frontend/browser/`

Test Orientation

Run frontend checks from the frontend folder when available:

```
npm run build  
npm run runtimes:check
```

Start here when chat routes, config tabs, credentials, theme, source rendering, page image viewing, Kernel progress boxes or agent prompt behavior is wrong.

SampleDB

TYPE	LOCATION
bundled product/demo payload	SampleDB/

Role

SampleDB contains the bundled default demo Artifact Tree and Corpus DB. It is there so a fresh installation can show a working corpus immediately without the user first creating a taxonomy and ingesting documents.

Owns

- demo Artifact Tree content
- demo Corpus DB
- demo README/license notes

Consumes

- Corpus Builder DB compatibility
- installer packaging rules

Produces

- immediate post-install demo state
- sample evidence for user exploration

Must Not Own

- generated release output
- user-created production corpora
- installer logic
- test state

Implementation Anchors

- SampleDB/README.md
- SampleDB/Consciousness Travel - Default Demo/

Root Tooling

TYPE	LOCATION
repository support surface	tools/

Role

Root tooling supports development, runtime building, installer staging, test dispatch, release export and packaging helpers. It is not a runtime product module, but maintainers will use it constantly.

Owns

- all-in-one build scripts
- runtime build helpers
- installer staging helpers
- root test dispatch helpers
- portable runtime utilities
- semantic release export helpers
- Test Dojo draft QA control-plane material

Produces

- staged installer inputs
- runtime build artifacts
- test execution output
- release packaging metadata

Must Not Own

- module runtime state
- Corpus DB materialization
- Semantic Release authoring truth
- production user config

Implementation Anchors

- `tools/build-all-in-one-installer.py`
- `tools/all_in_one_build.py`
- `tools/all_in_one_stage.py`

- `tools/run-dev-tests.py`
- `tools/build-runtimes.py`
- `tools/installer_stage*.py`
- `tools/portable_runtime*.py`
- `tools/test-dojoo/`
- `run-dev-tests.bat`
- `pytest.ini`

Installer And Packaging

TYPE release packaging surface	SOURCE LOCATION installer/ and root build scripts
GENERATED OUTPUT dist/	

Role

The installer surface builds the Windows all-in-one installer. It stages source modules, runtimes, shortcuts and bundled payloads into a release layout and then produces the installer output.

Owns

- Inno Setup source
- all-in-one installer build entry
- staged release layout rules
- uninstall script generation
- shortcut and install metadata
- packaging filters

Produces

- dist/all-in-one/stage/
- installer executable output
- release manifests

Must Not Own

- module source truth
- generated runtime state as source
- user-created corpora
- local developer caches

Implementation Anchors

- build-all-in-one-installer.bat
- installer/OntologyMachineAllInOne.iss
- tools/build-all-in-one-installer.py

- `tools/all_in_one_config.py`
- `tools/all_in_one_stage.py`
- `tools/all_in_one_uninstall_script.py`
- `tools/installer_stage*.py`

`dist/` should be treated as generated release output. It is useful for verification and shipping, but not as authoritative source.

Cross-References, Not Module Cards

These surfaces are important, but they are not product modules:

Surface	Treatment
The Machine Doku/	documentation set and handover handbook
Semantic Kernel SPEC/	specification source for Kernel workflows and tool semantics
SPEC_Handover_Blueprint.md	architecture and working-boundary rulebook
FIELD_READY_AUDIT.md , FIELD_READY_FINDINGS.jsonl , FIELD_READY_DEBUG_PACKETS/	field-readiness audit evidence
Website.md	website/content planning
The_Ontology_Machine_Full_Documentation.md	older master outline/source document for the modular docs
.gitignore , .stignore	repository sync and ignore policy

Generated/local-state surfaces should not be cataloged as source modules:

Surface	Treatment
dist/	generated packaging output
state-dir/	local runtime/test state
.pytest_cache/	generated test cache
__pycache__ / , temporary wheelhouses, suite folders , .venv	generated support/cache material

Fast Module Selection Guide

If The Problem Is...	Start In
pipeline run starts, stops, hangs, counts or final notices	00 - Orchestrator
file opening, rendering, OCR, page images or raw extraction	01 - Optimizer
structured LLM meaning before validation	02 - Interpreter
validation failures, weak review flags or missing review flags	03 - Validator
taxonomy, projections, normalized payloads or release fingerprints	04 - Normalizer
Corpus DB schema, rows, embeddings, search, merge, rebuild or Base Graph	05 - Corpus Builder
advanced config/edit UI and owner surface rendering	06 - Edit Suite
MCP tool transport, tool visibility or owner delegation	07 - MCP Server
workflow dialogs, blockers, resume, recovery or receipts	08 - Semantic Control Kernel
chat UI, config UI, credentials, source links, page viewer or agent prompts	Client Frontend
demo corpus payload	SampleDB
installer, runtime build, root test dispatcher	tools / installer

Related Chapters

- [System Overview](#)
- [Architecture Map](#)
- [Contract Library](#)
- [Workflow Catalog](#)
- [Artifact Tree Guide](#)
- [Database Documentation](#)
- [Agent Documentation](#)